KU LEUVEN

FACULTEIT INDUSTRIËLE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Labo 4

Labo Geavanceerde Computerarchitectuur

Kjell NAESSENS
Beau FORREZ

Labo verantwoordelijke: T. Feys

Academiejaar 2025-2026

1 Context

In deze opdracht voeren we GPU-bewerkingen uit op afbeeldingen en matrices. Bij de afbeelding inverteren we de rode kleurcomponent en vergelijken we twee geheugenindelingen: interleaved RGB (RGBRGB...) en planar RGB (RR...RGG...GBB...) om het effect van coalesced versus un-coalesced geheugen te bekijken.

Bij de matrices implementeren we matrixvermenigvuldiging met gebruik van alleen global memory, shared memory en constant memory om de invloed van geheugen op de uitvoeringstijd te onderzoeken.

2 Methodologie

2.1 Afbeelding inverteren

De rode kleurcomponent wordt omgekeerd via $R_{nieuw} = 255 - R_{oud}$. We gebruiken twee geheugenindelingen:

- Interleaved RGB: R, G, B per pixel $\rightarrow$ un-coalesced.
- Planar RGB: alle R, daarna G, daarna B $\rightarrow$ coalesced.

We testen verschillende block-groottes (64, 128, 256, ..., 8192 threads) en meten de uitvoeringstijd.

2.2 Matrixvermenigvuldiging

We implementeren drie kernels voor $C = A \cdot B$:

- Global memory: threads lezen A en B uit global memory.
- Shared memory: blokken van A en B in shared memory voor hergebruik.
- Constant memory: B in constant memory, A blijft in global memory.

We meten de uitvoeringstijd met CUDA-events en variëren grid- en block-groottes voor kleine matrices.

3 Observaties en resultaten

3.1 Coalesced memory access

In Figuur 1 zien we dat coalesced memory (planar) telkens vluggere uitvoertijden heeft dan non-coalesced memory. Bij het aanpassen van de block size merken we dat de uitvoeringstijd ook

telkens verkleint bij toenemende block size. Na bepaalde block size zien we wel dat de uitvoeringstijd niet meer veel verandert.

```
student@B230-PC07:/media/student/beaufor/CompArch/L4$ ./test
Block size: 64 | Interleaved: 0.158176 ms | Planar: 0.130368 ms
Block size: 128 | Interleaved: 0.134496 ms | Planar: 0.110752 ms
Block size: 256 | Interleaved: 0.13424 ms | Planar: 0.110944 ms
Block size: 512 | Interleaved: 0.140128 ms | Planar: 0.112064 ms
Block size: 1024 | Interleaved: 0.145472 ms | Planar: 0.114336 ms
Block size: 2048 | Interleaved: 0.0016 ms | Planar: 0.0016 ms
Block size: 4096 | Interleaved: 0.001696 ms | Planar: 0.0016 ms
Block size: 8192 | Interleaved: 0.0016 ms | Planar: 0.001664 ms
```

Figuur 1: Afbeelding inverteren met (niet-)coalesced geheugen

3.2 Global, shared en constant memory

Figuur 2 toont het effect van verschillende geheugentypes. We zien gelijkaardige performantie bij global en shared memory, en de slechtste performantie bij constant memory.

```
student@B230-PC07:/media/student/beaufor/CompArch/L4$ ./test

===== Matrix size 16x16 =====
Tile 8 | Global: 0.00447104 ms | Shared: 0.00473024 ms | Constant: 0.00469504 ms | Max diff: 0
Tile 16 | Global: 0.00396864 ms | Shared: 0.00414272 ms | Constant: 0.00742528 ms | Max diff: 0

===== Matrix size 32x32 =====
Tile 8 | Global: 0.00531968 ms | Shared: 0.0057568 ms | Constant: 0.00638464 ms | Max diff: 0
Tile 16 | Global: 0.00491456 ms | Shared: 0.00538816 ms | Constant: 0.0106822 ms | Max diff: 0
Tile 32 | Global: 0.00707136 ms | Shared: 0.00679168 ms | Constant: 0.0571923 ms | Max diff: 0

===== Matrix size 48x48 =====
Tile 8 | Global: 0.00810688 ms | Shared: 0.00877632 ms | Constant: 0.0122323 ms | Max diff: 0
Tile 16 | Global: 0.0066176 ms | Shared: 0.00700672 ms | Constant: 0.0238707 ms | Max diff: 0
Tile 32 | Global: 0.00904448 ms | Shared: 0.0102912 ms | Constant: 0.0836282 ms | Max diff: 0

===== Matrix size 64x64 =====
Tile 8 | Global: 0.0156275 ms | Shared: 0.0173517 ms | Constant: 0.0327437 ms | Max diff: 0
Tile 16 | Global: 0.0122694 ms | Shared: 0.0114362 ms | Constant: 0.0561216 ms | Max diff: 0
Tile 32 | Global: 0.0111296 ms | Shared: 0.0104 ms | Constant: 0.115993 ms | Max diff: 0
Tile 64 | Global: 0.00163904 ms | Shared: 0.00164864 ms | Constant: 0.00165504 ms | Max diff: 0

===== Matrix size 80x80 =====
Tile 8 | Global: 0.0225101 ms | Shared: 0.0280774 ms | Constant: 0.0501139 ms | Max diff: 0
Tile 16 | Global: 0.0168275 ms | Shared: 0.017353 ms | Constant: 0.087289 ms | Max diff: 0
Tile 32 | Global: 0.0186125 ms | Shared: 0.0238701 ms | Constant: 0.170442 ms | Max diff: 0
Tile 64 | Global: 0.001648 ms | Shared: 0.00165504 ms | Constant: 0.00165376 ms | Max diff: 0

===== Matrix size 96x96 =====
Tile 8 | Global: 0.0436262 ms | Shared: 0.0452928 ms | Constant: 0.0950854 ms | Max diff: 0
Tile 16 | Global: 0.028631 ms | Shared: 0.024521 ms | Constant: 0.191036 ms | Max diff: 0
Tile 32 | Global: 0.0271872 ms | Shared: 0.0243501 ms | Constant: 0.331225 ms | Max diff: 0
Tile 64 | Global: 0.00162816 ms | Shared: 0.00162048 ms | Constant: 0.00163136 ms | Max diff: 0
```

Figuur 2: Matrixvermenigvuldiging met types geheugen

4 Besluit

4.1 Vragen

4.2 Part 1: Coalesced memory access

4.2.1 Wat is het prestatieverschil tussen de coalesced en niet-coalesced kernel?

De coalesced (planar) kernel is sneller dan de niet-coalesced (interleaved) kernel, omdat opeenvolgende threads opeenvolgende geheugenadressen lezen, waardoor de memory accesses tegelijk als een 'burst' worden uitgevoerd.

4.2.2 Hoe verandert het prestatieverschil afhankelijk van de input en/of grid grootte?

Bij grotere block sizes zien we betere prestaties. De uitvoeringstijd neemt af bij toenemende block sizes, hoewel er vanaf bepaalde block sizes geen significante verbeteringen meer zijn. Er is een grote toename van performantie wanneer we full warps gebruiken in plaats van partial warps.

4.3 Part 2: Global, shared, en constant memory

4.3.1 Hoeveel tijd kost het overzetten van geheugen (van CPU naar GPU globaal; GPU globaal naar GPU shared)?

Het overzetten van de matrices van CPU naar GPU globaal geheugen duurde gemiddeld 0.016 ms voor een matrix van 120x120, terwijl het overzetten van de matrices van GPU globaal naar GPU shared gemiddeld 0.0024 ms duurde, en ook afnam voor grotere tiles (waardoor er minder geheugen transfers gebeurden). Deze laatste geheugenverplaatsing gebeurt ook binnen de GPU (binnen eenzelfde device) terwijl de verplaatsing van CPU naar GPU over de PCIe moet gebeuren en daardoor duurder is.

4.3.2 Hoe verhouden de uitvoeringstijden van de matrixvermenigvuldiging zich tot elkaar bij het gebruik van de drie verschillende geheugentypes?

De uitvoeringstijd bij gebruik van global en shared memory zijn over het algemeen zeer gelijkaardig, we merken geen opvallende verschillen. De tijd bij het gebruik van constant memory is echter een pak trager dan deze voorgaande geheugentypes.

4.3.3 Hoe schalen de uitvoeringstijden met de input en/of de grid grootte?

De uitvoeringstijden stijgen mee met de inputgrootte, hoewel grotere tiles hier minder last van hebben. Bij grote matrices zorgen grotere tiles namelijk voor betere efficiëntie omdat er minder geheugenaccesses zijn. De uitvoeringstijd neemt ook af bij grotere tiles, wat duidelijk te zien is bij deze grotere matrices.

5 Appendix

5.1 Part 1: Coalesced memory access

```

#include <iostream>
#include <cuda_runtime.h>
#include <cstdint>
#include <cstdlib>
#include <cstdio>

#define M 960       // width
#define N 1280      // height
#define C 3         // RGB
#define OFFSET 16   // PPM header offset

inline void cudaCheck(cudaError_t err) {
    if (err != cudaSuccess) {
        std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
        exit(EXIT_FAILURE);
    }
}

// ---- READ .ppm ----
uint8_t* get_image_array() {
    FILE *imageFile = fopen("anna.ppm", "rb");
    if (!imageFile) { perror("ERROR opening file"); exit(EXIT_FAILURE); }
    uint8_t header[OFFSET]; fread(header, 1, OFFSET, imageFile);
    uint8_t* img = (uint8_t*)malloc(M*N*C);
    fread(img, 1, M*N*C, imageFile);
    fclose(imageFile);
    return img;
}

// ---- WRITE .ppm ----
void save_image_array(uint8_t* img, const char* filename) {
    FILE *f = fopen(filename, "wb");
    fprintf(f, "P6\n%d %d\n255\n", M, N);
    fwrite(img, 1, M*N*C, f);
    fclose(f);
}

// ---- KERNEL: Interleaved RGB (un-coalesced) ----
__global__ void invert_red_interleaved(uint8_t* img) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int totalPixels = M*N;
    if (idx >= totalPixels) return;
    int rIdx = idx * C;       // R-component
    img[rIdx] = 255 - img[rIdx];
}
```

```
47
48 // ---- KERNEL: Planar RGB (coalesced) ----
49 __global__ void invert_red_planar(uint8_t* img) {
50     int idx = blockIdx.x * blockDim.x + threadIdx.x;
51     int totalPixels = M*N;
52     if (idx >= totalPixels) return;
53     img[idx] = 255 - img[idx];  // R-plane
54 }
55
56 // ---- Convert interleaved -> planar ----
57 uint8_t* convert_to_planar(const uint8_t* interleaved) {
58     int pixels = M*N;
59     uint8_t* planar = (uint8_t*)malloc(pixels*C);
60     for(int i=0;i<pixels;i++){
61         planar[i] = interleaved[i*C];         // R
62         planar[i+pixels] = interleaved[i*C+1]; // G
63         planar[i+2*pixels] = interleaved[i*C+2]; // B
64     }
65     return planar;
66 }
67
68 // ---- Convert planar -> interleaved ----
69 uint8_t* convert_to_interleaved(const uint8_t* planar) {
70     int pixels = M*N;
71     uint8_t* interleaved = (uint8_t*)malloc(pixels*C);
72     for(int i=0;i<pixels;i++){
73         interleaved[i*C]     = planar[i];          // R
74         interleaved[i*C+1]   = planar[i+pixels];   // G
75         interleaved[i*C+2]   = planar[i+2*pixels]; // B
76     }
77     return interleaved;
78 }
79
80 // ---- MAIN ----
81 int main() {
82     const int pixels = M*N;
83     const size_t bytes = pixels*C*sizeof(uint8_t);
84
85     uint8_t* h_in = get_image_array();
86     uint8_t* h_planar_in = convert_to_planar(h_in);
87
88     uint8_t *d_in;
89     cudaCheck(cudaMalloc(&d_in, bytes));
90
91     cudaEvent_t start, stop;
92     cudaEventCreate(&start);
93     cudaEventCreate(&stop);
94
95     int blockSizes[] = {64, 128, 256, 512, 1024, 2048, 4096, 8192};
96     int numBlockSizes = 8;
97
```

```
    for(int b=0; b<numBlockSizes; b++){
        int threads = blockSizes[b];
        int blocks = (pixels + threads - 1)/threads;

        // ---- Interleaved ----
        cudaCheck(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));
        cudaEventRecord(start);
        invert_red_interleaved<<<blocks, threads>>>(d_in);
        cudaEventRecord(stop);
        cudaCheck(cudaEventSynchronize(stop));
        float t_interleaved = 0;
        cudaEventElapsedTime(&t_interleaved, start, stop);

        if(b == 2){
            uint8_t* h_interleaved_out = (uint8_t*)malloc(bytes);
            cudaCheck(cudaMemcpy(h_interleaved_out, d_in, bytes,
                cudaMemcpyDeviceToHost));
            save_image_array(h_interleaved_out, "output_red_interleaved.ppm");
            free(h_interleaved_out);
        }

        // ---- Planar ----
        cudaCheck(cudaMemcpy(d_in, h_planar_in, bytes, cudaMemcpyHostToDevice))
            ;
        cudaEventRecord(start);
        invert_red_planar<<<blocks, threads>>>(d_in);
        cudaEventRecord(stop);
        cudaCheck(cudaEventSynchronize(stop));
        float t_planar = 0;
        cudaEventElapsedTime(&t_planar, start, stop);

        if(b == 2){
            uint8_t* h_planar_out = (uint8_t*)malloc(bytes);
            cudaCheck(cudaMemcpy(h_planar_out, d_in, bytes,
                cudaMemcpyDeviceToHost));
            uint8_t* h_planar_interleaved = convert_to_interleaved(h_planar_out
                );
            save_image_array(h_planar_interleaved, "output_red_planar.ppm");
            free(h_planar_out);
            free(h_planar_interleaved);
        }

        std::cout << "Block size: " << threads
                  << " | Interleaved: " << t_interleaved
                  << " ms | Planar: " << t_planar << " ms\n";
    }

    // ---- Cleanup ----
    cudaFree(d_in);
    free(h_in);
    free(h_planar_in);
```

```
    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return 0;
}
```

5.2 Part 2: Global, shared, en constant memory

```

#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib>

#define N 120    // matrix size N x N (small for debugging)
#define RUNS 100 // aantal herhalingen voor gemiddelde

inline void cudaCheck(cudaError_t err){
    if(err != cudaSuccess){
        std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
        exit(EXIT_FAILURE);
    }
}

__global__ void matMulGlobal(float* A, float* B, float* C, int n){
    int row = blockIdx.y*blockDim.y + threadIdx.y;
    int col = blockIdx.x*blockDim.x + threadIdx.x;
    if(row < n && col < n){
        float sum = 0.0f;
        for(int k=0;k<n;k++)
            sum += A[row*n + k] * B[k*n + col];
        C[row*n + col] = sum;
    }
}

__global__ void matMulShared(float* A, float* B, float* C, int n){
    __shared__ float sA[32][32];
    __shared__ float sB[32][32];

    int row = blockIdx.y*blockDim.y + threadIdx.y;
    int col = blockIdx.x*blockDim.x + threadIdx.x;
    float sum = 0.0f;

    int TILE = blockDim.x;
    for(int t=0; t<(n+TILE-1)/TILE; t++){
        if(row<n && t*TILE + threadIdx.x<n)
            sA[threadIdx.y][threadIdx.x] = A[row*n + t*TILE + threadIdx.x];
        else
            sA[threadIdx.y][threadIdx.x] = 0.0f;

```

```
        if(col<n && t*TILE + threadIdx.y<n)
            sB[threadIdx.y][threadIdx.x] = B[(t*TILE + threadIdx.y)*n + col];
        else
            sB[threadIdx.y][threadIdx.x] = 0.0f;

        __syncthreads();

        for(int k=0;k<TILE;k++)
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];

        __syncthreads();
    }

    if(row<n && col<n)
        C[row*n + col] = sum;
}

__constant__ float cB[N*N];
__global__ void matMulConstant(float* A, float* C, int n){
    int row = blockIdx.y*blockDim.y + threadIdx.y;
    int col = blockIdx.x*blockDim.x + threadIdx.x;
    if(row<n && col<n){
        float sum = 0.0f;
        for(int k=0;k<n;k++)
            sum += A[row*n + k] * cB[k*n + col];
        C[row*n + col] = sum;
    }
}

int main(){
    int size = N*N;
    size_t bytes = size * sizeof(float);

    float *hA = (float*)malloc(bytes);
    float *hB = (float*)malloc(bytes);
    float *hC = (float*)malloc(bytes);
    for(int i=0;i<size;i++){ hA[i]=1.0f; hB[i]=1.0f; }

    float *dA, *dB, *dC;
    cudaCheck(cudaMalloc(&dA, bytes));
    cudaCheck(cudaMalloc(&dB, bytes));
    cudaCheck(cudaMalloc(&dC, bytes));
    cudaCheck(cudaMemcpy(dA, hA, bytes, cudaMemcpyHostToDevice));
    cudaCheck(cudaMemcpy(dB, hB, bytes, cudaMemcpyHostToDevice));
    cudaCheck(cudaMemcpyToSymbol(cB, hB, bytes));

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    int blockSizes[] = {8, 16, 32, 64, 128};
```

```
    int numBlockSizes = 5;

    for(int b=0;b<numBlockSizes;b++){
        int TILE = blockSizes[b];
        dim3 threads(TILE, TILE);
        dim3 blocks((N+TILE-1)/TILE, (N+TILE-1)/TILE);

        float totalGlobal=0, totalShared=0, totalConst=0;

        for(int i=0;i<RUNS;i++){
            float tGlobal, tShared, tConst;

            // Global
            cudaEventRecord(start);
            matMulGlobal<<<blocks, threads>>>(dA, dB, dC, N);
            cudaEventRecord(stop);
            cudaCheck(cudaEventSynchronize(stop));
            cudaEventElapsedTime(&tGlobal, start, stop);

            // Shared
            cudaEventRecord(start);
            matMulShared<<<blocks, threads>>>(dA, dB, dC, N);
            cudaEventRecord(stop);
            cudaCheck(cudaEventSynchronize(stop));
            cudaEventElapsedTime(&tShared, start, stop);

            // Constant
            cudaEventRecord(start);
            matMulConstant<<<blocks, threads>>>(dA, dC, N);
            cudaEventRecord(stop);
            cudaCheck(cudaEventSynchronize(stop));
            cudaEventElapsedTime(&tConst, start, stop);

            totalGlobal += tGlobal;
            totalShared += tShared;
            totalConst += tConst;
        }

        std::cout << "Tile size: " << TILE
                  << " | Global avg: " << totalGlobal/RUNS << " ms"
                  << " | Shared avg: " << totalShared/RUNS << " ms"
                  << " | Constant avg: " << totalConst/RUNS << " ms\n";
    }

    cudaCheck(cudaMemcpy(hC, dC, bytes, cudaMemcpyDeviceToHost));

    cudaFree(dA); cudaFree(dB); cudaFree(dC);
    free(hA); free(hB); free(hC);
    cudaEventDestroy(start); cudaEventDestroy(stop);

    return 0;
```

```
144 }
```